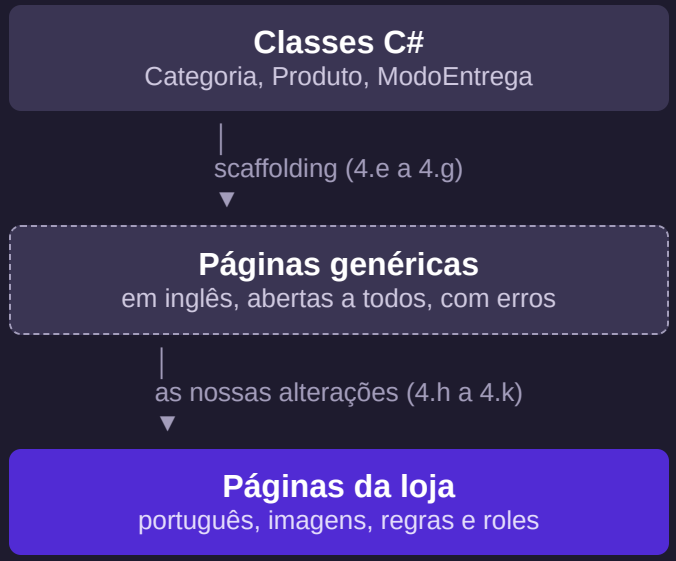
# Páginas geradas vs. páginas da loja

O scaffolding desenha a planta da casa: uma divisão por entidade, uma porta por operação. Funciona, mas é genérica.

Nós pomos as fechaduras, as placas em português e tudo o que a planta não previa. Estes slides mostram cada alteração: **o antes, o depois e o porquê**.



ALÍNEAS 4.E A 4.L · VISÃO GERAL

# Nove diferenças entre o gerado e o nosso

| O quê | O scaffolding gera | O que nós fizemos | Porquê |
|---|---|---|---|
| **Imagem (`byte[]`)** | `<InputText>` ligado a bytes: não compila | `<InputFile>` + `IImagensService` | Uma imagem escolhe-se do disco, não se escreve |
| **Chave estrangeira** | `<InputNumber>`: escreve-se o número | `<InputSelect>` com os nomes | O utilizador pensa em nomes; a tabela guarda o Id |
| **Listagem de produtos** | Sem o nome da categoria | `Include(p => p.categoria)` | Sem `Include`, `p.categoria` vem vazio (`null`) |
| **Fábrica de contextos** | Não a regista no `Program.cs` | `AddDbContextFactory` | As páginas pedem `IDbContextFactory` com `@inject` |
| **Apagar ModoEntrega** | `Remove` direto: erro REFERENCE | `Include` dos produtos antes do `Remove` | O EF Core só põe `NULL` nos produtos que carregou |
| **Apagar Categoria** | Apaga logo, sem avisar | `<Confirmacao>` com o n.º de produtos | `ON DELETE CASCADE` apaga também os produtos |
| **Quem pode entrar** | Qualquer pessoa, mesmo sem login | `[Authorize(Roles = "Admin,Gestor")]` | Gerir a loja é só para quem a gere |
| **Depois de gravar** | Volta à lista sem dizer nada | `<Aviso />` a verde ou a vermelho | Quem usa tem de saber se correu bem |
| **Textos** | Em inglês: Create New, Back to List | Em português | Os utilizadores da loja falam português |

**Regra de bolso:** o scaffolding sabe **que colunas existem** (lê as classes). Não sabe **as regras da nossa loja**: quem entra, o que acontece ao apagar, como se mostra uma imagem. Isso é connosco.

ALÍNEAS 4.E A 4.I · IMAGEM

# Imagem: de caixa de texto a «Escolher ficheiro»

GERADO

CategoriaPages/Create.razor, tal como é gerado

```
<div class="mb-3">
  <InputText @bind-Value="Categoria.Imagem" />
</div>
```

Lista de Erros

```
CS1503 não é possível converter de 'byte[]' para 'string'
```

## Para totós

Uma caixa de texto serve para escrever letras. Uma imagem são milhares de bytes (números de 0 a 255): ninguém os escreve à mão.

O scaffolding não tem componente para `byte[]` e usa o de texto, por defeito. Daí o erro.

DEPOIS DE NÓS

CategoriaPages/Create.razor (excerto)

```
<InputFile OnChange="CarregaImagem" />
<img src="@Imagens.Src(Categoria.Imagem, ...)" />

// no @code, quando se escolhe o ficheiro
erroImagem = Imagens.Valida(e.File.Name, e.File.Size);
Categoria.Imagem = await Imagens.LerBytesAsync(conteudo);
Categoria.UrlImagem = e.File.Name;
```

## Porquê assim

- `<InputFile>` abre a janela «Escolher ficheiro» do sistema.
- O `IImagensService` valida e lê os bytes. A regra fica escrita uma vez e serve as 10 páginas.
- Ainda não grava: a pré-visualização muda logo, o `INSERT` só acontece no Gravar.

# Categoria: escrever o número ou escolher o nome

GERADO

ProdutoPages/Create.razor, tal como é gerado

```
<InputNumber @bind-Value="Produto.CategoriaId" />
```

CategoriaId

3

E o 3 é o quê? Fruta? Legumes? E se alguém escrever 99, uma categoria que não existe? A base de dados recusa, e o utilizador não percebe porquê.

DEPOIS DE NÓS

ProdutoPages/Create.razor (excerto)

```
<InputSelect @bind-Value="Produto.CategoriaId">
  <option value="0">-- escolha --</option>
  @foreach (var c in categorias)
  { <option value="@c.Id">@c.Nome</option> }
</InputSelect>
```

Categoria

Legumes ▼

Só aparecem categorias que existem. Com `-- escolha --` (valor 0), a página recusa gravar sem categoria.

**O utilizador vê** o nome: «Legumes» → **`value`** = o Id da categoria → **A tabela Produtos** guarda o Id em `CategoriaId`

ALÍNEAS 4.I E 4.K · APAGAR

# Apagar: o que a página gerada não sabe

## ModoEntrega (relação opcional)

Gerado: apaga o objeto que já tinha lido

```
context.ModosEntrega.Remove(modoEntrega);
await context.SaveChangesAsync();
✗ The DELETE statement conflicted with
  the REFERENCE constraint ...
```

Nosso: carrega os produtos antes de apagar

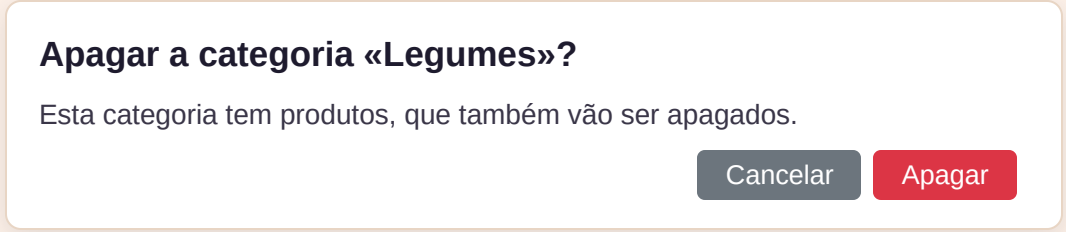
```
var aApagar = await context.ModosEntrega
    .Include(m => m.produtos)
    .FirstOrDefaultAsync(m => m.Id == Id);
context.ModosEntrega.Remove(aApagar);
```

**Para totós:** os produtos têm um «post-it» com o modo de entrega. O SQL Server não deixa apagar o modo com post-its ainda colados. O `Include` traz esses produtos; o EF Core descola os post-its (`ModoEntregaId = NULL`) e só depois apaga.

## Categoria (relação obrigatória)

**Gerado:** apaga logo. Como a ficha 4 criou `ON DELETE CASCADE`, o SQL Server apaga também **todos os produtos** dessa categoria. Sem aviso, sem volta atrás.

**Nosso:** a página conta os produtos e mostra a `<Confirmacao>`.




Exemplo do género de janela. O `Cancelar` não apaga nada.

ALÍNEA 4.H · PROGRAM.CS

# Program.cs: a fábrica que o scaffolding não regista

| | | | | |
|---|---|---|---|---|
| **Antes** | A página pede `IDbContextFactory` | → | O `Program.cs` não registou nenhuma | → | × Erro ao abrir a página: `no registered service of type IDbContextFactory` |
| **Depois** | A página pede `IDbContextFactory` | → | O `Program.cs` registou a fábrica | → | ✓ Recebe a fábrica; cada operação cria o seu contexto e larga-o no fim. |

ANTES (TEMPLATE DA FICHA 3)

```
builder.Services.AddDbContext<ApplicationDbContext>(
    options => options.UseSqlServer(connectionString));
```

DEPOIS DE NÓS

```
builder.Services.AddDbContextFactory<ApplicationDbContext>(
    options => options.UseSqlServer(connectionString));
```

## Porque é que o scaffolding não o faz

Gera páginas que pedem a fábrica e acrescenta linhas ao `Program.cs`, mas, como o contexto já existia, não mexe no registo dele. A primeira página que se abre dá o erro.

## Para totós

A página chega ao supermercado e pede um carrinho, mas ninguém pôs carrinhos à entrada. Com a fábrica há uma fila deles: cada operação tira um e larga-o no fim. O Identity continua a ter o seu `ApplicationDbContext`.

ALÍNEAS 4.I A 4.K · ROLES

# Quem entra: de porta aberta a porta com fechadura

GERADO: O TOPO DA PÁGINA

ProdutoPages/Create.razor

```
@page "/produtos/create"
@using Microsoft.EntityFrameworkCore
@using GestaoLoja.Entities
@* nenhuma regra de acesso *@
```

DEPOIS DE NÓS: MAIS UMA LINHA

ProdutoPages/Create.razor (e as outras 14)

```
@page "/produtos/create"
@using Microsoft.AspNetCore.Authorization
...
@attribute [Authorize(Roles = "Admin,Gestor")]
```

**Experiência:** quatro pessoas escrevem no browser `https://localhost:7166/produtos/create`

| Quem | Sem [Authorize] (gerado) | Com [Authorize(Roles = "Admin,Gestor")] |
|---|---|---|
| **Sem sessão iniciada** | Entra e cria produtos | Vai para o Login |
| **Cliente** | Entra e cria produtos | Página «acesso negado» |
| **Gestor** | Entra | Entra |
| **Admin** | Entra | Entra |

**Esconder não é proteger:** tirar a entrada do menu (`AuthorizeView`) não impede ninguém de escrever o endereço. Quem fecha a porta é o `[Authorize]`. A vírgula em `"Admin,Gestor"` é um OU.

ALÍNEAS 4.I A 4.K · ACABAMENTOS

# Mensagens, imagens na lista e português

GERADO

Grava e volta à lista calado. Se falhar (nome repetido), dá um erro que ninguém percebe.

## Mensagens: `<Aviso />`

Categoria criada.

Já existe uma categoria com esse nome.

**Porquê:** como no multibanco, que diz «Operação concluída». Sem isso fica a dúvida, e carrega-se em Gravar outra vez.

GERADO

Listagem só com texto: nenhuma imagem à vista.

## Imagens na listagem

```
<TemplateColumn Context="categoria">
  <img src="@Imagens.Src(...)" />
</TemplateColumn>
```

**Porquê:** numa frutaria reconhece-se o produto pela imagem antes de ler o nome. O `PropertyColumn` só mostra texto; o `TemplateColumn` deixa pôr HTML à medida.

GERADO

Create New · Back to List · Edit | Details | Delete

## Português

| | |
|---|---|
| Create New | → Criar novo |
| Back to List | → Voltar à lista |
| Edit · Details · Delete | → Editar · Detalhes · Apagar |

**Porquê:** quem usa a Gestão de Loja fala português. Botões em inglês fazem a aplicação parecer inacabada.

PARA LEVAR DA AULA

# O scaffolding sabe as colunas; não sabe a loja

### 1. Corrigir

O que não compila ou falha ao usar:

- a `Imagem` (`byte[]`)
- a fábrica no `Program.cs`
- apagar um ModoEntrega

### 2. Completar

O que falta para ser usável:

- caixas de escolha e `Include`
- confirmação antes de apagar
- mensagens, imagens e português

### 3. Proteger

O que decide quem entra:

- `[Authorize(Roles = "Admin,Gestor")]` nas 15 páginas
- `AuthorizeView` só esconde

Gerar poupa a escrita repetida: 5 páginas × 3 entidades = 15 páginas. O trabalho de quem programa é o que vem a seguir.